arXiv:1410.1387v1 [math.NA] 2 Oct 2014

High-Order Finite-differences on multi-threaded architectures using OCCA*

David S. Medina[1], Amik St-Cyr[2], and Timothy Warburton[1]

[1] Rice University, Computational and Applied Mathematics,
{dsm5,tim.warburton}@rice.edu
[2] Royal Dutch Shell, Seismic applications team, amik.st-cyr@shell.com

Abstract. High-order finite-difference methods are commonly used in wave propagators for industrial subsurface imaging algorithms. Computational aspects of the reduced linear elastic vertical transversely isotropic propagator are considered. Thread parallel algorithms suitable for implementing this propagator on multi-core and many-core processing devices are introduced. Portability is addressed through the use of the OCCA runtime programming interface. Finally, performance results are shown for various architectures on a representative synthetic test case.

1 Introduction

High-order finite-differences are used in seismic imaging and many other industrial applications primarily because of their computational efficiency. High-order wave propagators lie at the heart of numerous seismic imaging applications, such as full waveform inversion and reverse time migration (RTM). We study multi-threaded performance on various current and emerging computing architectures of propagators for vertical transversely isotropic media (VTI).

The VTI propagator introduced in [3] is given by

$$\frac{\partial^2 p}{\partial t^2} = \nu_x^2 \left[\frac{\partial^2 p}{\partial x^2} + \frac{\partial^2 p}{\partial y^2} \right] + \nu_z^2 \frac{\partial^2 q}{\partial z^2} + s(t)\delta(\mathbf{x} - \mathbf{x_i}), \tag{1}$$

$$\frac{\partial^2 q}{\partial t^2} = \nu_n^2 \left[\frac{\partial^2 p}{\partial x^2} + \frac{\partial^2 p}{\partial y^2} \right] + \nu_z^2 \frac{\partial^2 q}{\partial z^2}. \tag{2}$$

In the preceding equations, p is an approximation for the P-wave while q is and auxiliary wavefield variable. ϵ and δ are the anisotropic parameters. The vertical P-wave velocity is represented with ν_z and its horizontal component is $\nu_x = \nu_z\sqrt{1+2\epsilon}$ while the normal move-out velocity is $\nu_n = \nu_z\sqrt{1+2\delta}$. For this approximation to be relevant $\epsilon - \delta \leq 0$ is necessary. The forcing considered in our benchmark is the Ricker wavelet $s = (1 - 2\pi^2 f^2 t^2)e^{-\pi^2 f^2 t^2}$ with $f = 15Hz$.

* Partial funding from Royal Dutch Shell, ONR award N00014-13-1-0873, and subcontract to the Argonne CESAR Exascale Co-design Center under award number ANL 1F-32301. Shell release of technical information: RTI 2094414 & 233820614.

We consider a centered finite-difference discretization in time and space in second order form on infinite domains. For $\mathbf{u}(\mathbf{x},t) = (p,q)^T$ and $\mathbf{F}(\mathbf{u},\mathbf{x},t)$ set as the right and side of (1)-(2) the centered in time approximation reads

$$\mathbf{u}^{n+1} - 2\mathbf{u}^n + \mathbf{u}^{n-1} \approx \Delta t^2 \mathbf{F}(\mathbf{u}^n) \tag{3}$$

where $\mathbf{u}^k \equiv \mathbf{u}(\mathbf{x}, t^n)$ with $t^n = n\Delta t$. High-order finite-difference stencils are of practical importance for the efficient numerical solutions of wave propagation problems [1,13]. Indeed, for a similar number of points composing the computational grid, the number of points required to resolve the shortest wavelength (as defined by Nyquist) decreases and gets close to the spectral or pseudo-spectral limit of two points per wavelength [4]. Most propagators used in seismic applications use two different flavors of high-order finite-differences. The earth subsurface is geologically horizontally layered. Since depth, represented by the z coordinate, will experience the most changes in the rock properties, while in the $x-y$ planes the properties will remain constant within a layer. Therefore, a common strategy is to have a symmetric stencil in the $x-y$ direction, while handling a variable spacing in z. The weights and spacings can be optimized to handle a variety of physical and numerical properties [6,5]. For simplicity, we suppose a domain $\Omega = [0, Lx] \times [0, Ly] \times [0, Lz]$ where $\Delta x = \Delta y = h$ and Δz_k result from the discretization in space using $N_{d=\{x,y,z\}}$ points in each direction respectively. The mesh size in the z direction varies per grid point belonging to a different $x-y$ plane. Adopting the convention $p(x_i, y_j, z_k) = p_{i,j,k}$, the differentiation stencil in the $x-y$ plane is

$$h^2\left(\frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2}\right)p_{i,j,k} \approx w_0^{xy} p_{i,j,k} + \sum_{l=1}^{R_{xy}} w_l^{xy} \left(p_{i+l,j,k} + p_{i-l,j,k} + p_{i,j+l,k} + p_{i,j-l,k}\right) \tag{4}$$

where the w_l^{xy} are the $R_{xy}+1$ weights for approximating the two dimensional Laplacian. The differentiation is a bit simpler in the z direction:

$$\frac{\partial^2}{\partial z^2} q_{i,j,k} \approx \sum_{l=-R_z+}^{R_z} w_{k,l}^z q_{i,j,k+l}. \tag{5}$$

Again, the $w_{k,l}^z$ are the weights for approximated the second derivative. However, for each position z_k where the value of the derivative is sought, $2R_z+1$ weights are needed instead of R_z+1 as in the symmetric case due to the asymmetry in the z direction. The grid size Δz_k is absorbed into the $w_{k,l}^z$ weights in practice and therefore are not appearing above. The domain Ω is embedded into a larger domain where a damping formula is applied as in [2]. Outside the damping region, the solution is assumed to be zero for R_{xy} points in the x and y directions and R_z points in z.

In the following sections, we describe the reduced elastic VTI model for isotropic media together with a typical finite-difference discretization employed in industry.

2 Computational efficiency of high-order finite differences

The peak parallel floating point operations per second (flops) available on modern CPUs and GPUs have followed the trend set by Moores law. Unfortunately, the available memory bandwidth lagged this trend. This gap in bandwidth currently favors algorithms generating lots of flops per byte of data moved [7]. For VTI, using this type of stencil, a pessimistic computational intensity is

$$CI \approx (1/4)(5R_{xy} + 4R_z)/(4R_{xy} + 2R_z) \approx 0.4 \text{ flops/byte} \tag{6}$$

where most of the loads are assumed to be not in cache. An idealized version is to consider the least loads as possible (assumes most of the data in cache). This is done by assuming three single precision loads per point for the model properties (ν_x^2, ν_n^2 and ν_z^2) as well as the two pairs of loads and stores for $\mathbf{u}^n$ and, respectively, $\mathbf{u}^{n+1}$.

$$CI \approx (1/28)(5R_{xy} + 4R_z) \approx 0.3(R_{xy} + R_z) \text{ flops/byte} \tag{7}$$

Therefore increasing the order of the stencil augments the intensity since the low order case is close to the pessimistic estimate. In practice, better approximations can be obtained [14]. Performing those measurements automatically using hardware counters is still in development. Moreover in [4], the effectiveness of finite-differences for wave propagation problems is shown to increase with order. Indeed for a fixed number of Fourier modes "M", $\tilde{N}_d$ points are required to guarantee their resolution according to Nyquist. The relation is approximated with $\tilde{N}_d = c_p M^{1+(2R)^{-1}}$ and therefore doubling the polynomial order for a fixed number of modes, leads to $M^{\frac{1}{4}}$ times more points in each direction. Since the method is explicit in time, the total increase in computational cost is M in 3D.

3 OCCA: portable multi-threading

OCCA, a recently developed C++ library for handling multi-threading is employed. It uses run-time compilation and macro expansions which results in a novel and simple single kernel language that expands to multiple threading languages. OCCA currently supports device kernel expansions for the OpenMP, OpenCL, pThreads, Intel COI and CUDA languages. Performance characteristics are given for our implementations built on top of the OCCA API. Using the unified OCCA programming approach allows customized kernels optimized for CPU and GPU architectures with a single "host" code.

The OCCA library is an API providing a kernel language and an abstraction layer to back-ends APIs such as OpenMP, OpenCL and CUDA see [11,12,10] amongst others. Although OCCA unites different threading platform back-ends, the main contributions is the abstraction of the kernel language. Using a macro-based approach, an OCCA kernel can be expanded at runtime to suit OpenMP with dynamic pragma insertions or a device kernel using either OpenCL or

CUDA and is taylored to streamline the incorporation of future threading languages with ease. This is similar to source to source front-ends to compilers (cite HMPP and PGI) however, the translation is at runtime and performed mostly by the c-preprocessor. With such an approach, the control over the optimization is completely left to the user and more information is available for the compiler.

OCCA host API: Aside from language-based libraries from OpenMP, OpenCL or CUDA, the OCCA host API is a stand-alone library.This independence allows OCCA to be combined with other libraries without conflict, as shown in 1. The three key components that influenced the OCCA host API development:

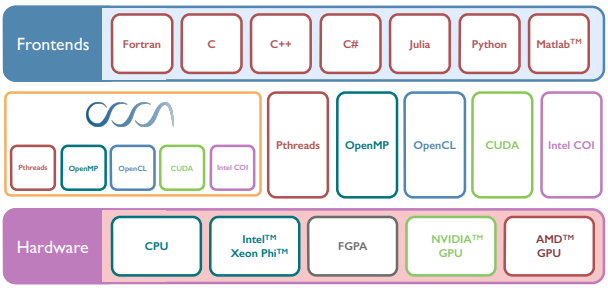

Fig. 1. OCCA wraps different language APIs and is non-conflicting with external libraries in either platform

the platform device, device memory and device kernels. Presenting the entire OCCA API is not feasible in this paper. For the complete details see [8] and the git repository for the latest developments[3]. We try here to expose the minimal knowledge required to write the VTI kernel.

OCCA device: An OCCA device acts as a layer of abstraction between the OCCA API and the API from supported languages. Due to the just-in-time code generation, the platform target can be chosen at run-time. Enabled platforms are managed at compile-time in the case of unsupported platforms on the compiled architecture. An OCCA device generates a self-contained context and command queue [[stream]] from a chosen device, being a socketed processor, GPU or other OpenCL supported devices such as a Xeon Phi or an FPGA. Asynchronous computations with multiple contexts can be achieved using multiple OCCA devices. A device object can allocate memory and compile kernels for the physical device.

OCCA memory: The OCCA memory class abstracts the different device memory handles and provide some useful information such as device array sizes. Although memory handling in OCCA facilitates host-device communication, the

[3] `http://github.com/tcew/OCCA`

management of reading and writing between host and device is left to the programmer for performance reasons. The dedicated device memory class allows the OCCA kernel class to manage communications between distinct memory types.

OCCA kernel: The OCCA kernel class unites device function handles with a single interface, whether for a function pointer (OpenMP), cl_kernel (OpenCL), or cuFunction (CUDA). When using the OpenCL and CUDA kernel handles, passing the arguments through their respective API is simple. These implicit work-group [[block]] and work-item [[thread]] sizes are passed by an argument in CPU-modes such as OpenMP.

OCCA kernel language: GPU computing involves many threads and the thread-space is logically decomposed into thread-blocks. Thread blocks are queued for execution onto the available multiprocessors. In general a GPU chip has more than a single multiprocessor and the choices for number of blocks and threads per blocks are dependent on the algorithm, resources available and the developer. Taking as example the CUDA API, a general loop can be written as in **Listing 1.1**.

```
for(int bZ = 0; bZ < gridDim.z; ++bZ){        // (1)
  for(int bY = 0; bY < gridDim.y; ++bY){
    for(int bX = 0; bX < gridDim.x; ++bX){
      // Shared memory is initialized here
      for(int tZ = 0; tZ < blockDim.z; ++tZ){  // (2)
        for(int tY = 0; tY < blockDim.y; ++tY){
          for(int tX = 0; tX < blockDim.x; ++tX){
             // Work here,  initialize register memory
}}}}}}
```

Listing 1.1. The expansion of the implicit for-loops found in CUDA kernels is displayed. Loop grouping (1) expands multi-dimensional work-groups [[blocks]] and loop grouping (2) expands multi-dimensional work-items [[threads]].

The first three for loops are going through all the blocks while the three innermost loops iterate on the individual threads per block.This is an implicit loop, one would never write such a thing in practice as it would destroy the thread concurrency and all achievable performance. OCCA replaces the loops with the correct constructs and gives the developer access to the global indexing using `occaGlobalId{0,1,2}` which is computed with `blockIdx` and `threadIdx`. The OCCA equivalent general loop is contained in listing 1.2.

```
occaOuterFor2{       // Loop grouping (1)
  occaOuterFor1{
    occaOuterFor0{
      // Shared memory defined here
      occaInnerFor2{ // Loop grouping (2)
        occaInnerFor1{
          occaInnerFor0{
            // work here
}}}}}}
```

Listing 1.2. The OCCA programming model mirrors GPU programming, where group loopings (1) and (2) refer to work-groups [[blocks]] and work-items [[threads]] respectively.

The use of shared memory is still available in OCCA since it is essential for many GPU optimized codes. Shared memory still acts as a scratchpad cache for GPU architectures but can be seen as a prefetch buffer for CPU-modes in OCCA.

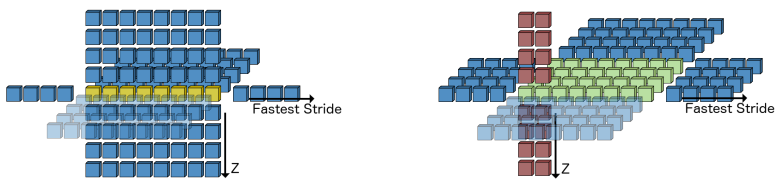

Fig. 2. The left panel represents a 3D finite-difference stencil vectorized with AVX. The fast stride is in the x direction and 8 single precision stencil evaluation are performed simultaneously. The right panel represents thread block with the large 2D subdomain the information loaded into shared (fast) memory. The register rolling in the z-direction is shown and for a "two-elements" kernel, each thread handles two columns.

The OCCA:OpenMP code performs the VTI steps uses the classic technique of cache blocking as seen in code listing 1.3. The best performing kernel had 2D cache blocking with the Z-block first followed by the Y-block. The innermost loop would be x (stride-1) then z and finally y. The z-blocks were handled in OCCA with `occaOuterFor2`, the y one with `occaOuterFor1` and so on. The vectorization was handled directly by the Intel compiler by placing a pragma `#pragma ivdep` in the `occaInnerFor0` (x) and making sure the data was correctly padded. The size of the blocking in z-y was determined as $(28, 20)$ by running the code over a set of grids and possible block ranges and comparing throughput times see section 4. The OpenMP first-touch policy was critical in obtaining performance across dual sockets as well as the correct thread affinity. Finally, to make sure the compiler was optimizing as depicted in Fig. 2, a hand written kernel with explicit register blocking was written: 5% increase in performance was observed.

```
Partition the top plane of the grid into B_x × B_y blocks of size w × h

For time-step n = 0, 1, ..., time-Steps
    For each block (b_i, b_j)                                   (1)
        For n = 0, 1, ..., N_z
            For each point (i, j, k) such that
                (b_i ≤ i < b_i + w) and (b_j ≤ j < b_j + h)         (2)
                Update p^{n+1}(i, j, k) and q^{n+1}(i, j, k)
            End For // Point Update
        End For     // Traversing depth
```

```
    End For          // Iterating over blocks
End For              // Computing a time-step update
```

Listing 1.3. For each time-step, the 2D blocks at the top of the structured grid sweep in the z direction and update all points in the current z plane.

A single implementation encompasses OCCA:OpenCL and OCCA:CUDA follows directly the work of [9]. As depicted in the right panel of Fig. 2, for a given thread block, the 2D $x - y$ stencil executes into fast shared memory while the z direction is handled by register rolling. If each thread handles one such column per thread block then this is a *one-element* approach while a *two-elements* approach consists of having two such columns per thread. Care was taken to align the data to enable coalescing loads to shared memory.

4 Performance

The VTI kernel is integrated in time for a thousand time steps. A metric of performance used in seismic is the throughput: number of sweeps through the entire grid block per second. The precision is set at $R_{xy} = 12$ and $R_z = 8$ and yields approximatively 92 flops per point. The CI optimistic model derived in section 2 yields a factor of 3.3.

Results on a dual socket node with E5-2670 are reported in table 1. The dual socket node is capable of 666 single precision GFlops while the bandwidth is 102.4 GB/s. The optimistic CI predicts a maximal peak of 47%. The results show the fastest OCCA kernel achieving 21% and good scalability as compared to the native OpenMP code (without OCCA). The difference stems from the added knowledge at compile time for OCCA, where all loop-bounds are known at compile time.

Project	Distribution	1 Thread	2 Threads	4 Threads	8 Threads	16 Threads	% Peak
Native	Compact	92	183 (98%)	360 (96%)	668 (89%)	1226 (82%)	17
Native	Scatter	92	183 (98%)	356 (95%)	686 (92%)	1191 (80%)	16
OCCA	Compact	115	229 (99%)	448 (97%)	820 (89%)	1548 (84%)	21
OCCA	Scatter	115	230 (100%)	454 (98%)	884 (96%)	1411 (76%)	19

Table 1. Multithreading scaling with OpenMP using alternative thread distributions on different number of cores (using two Xeon E5-2640 Processors)

Table 2 contains performance on GPU architectures that were based on optimized CUDA code and translated to OCCA. We note that performance seen in table 2 was on par with native code due to optimizations that can be done with run-time compilation including manual unrolling and manual bounds on OpenMP-loops.

Project	Kernel Language	K10 (1-chip)	K20x
Native	CUDA	1068	1440
Native (2)	CUDA	1296	2123
OCCA	OCCA:CUDA	1241	1934
OCCA (2)	OCCA:CUDA	1579	2431
OCCA	OCCA:OpenCL	1303	1954
OCCA (2)	OCCA:OpenCL	1505	2525

Table 2. Performance comparisons on the VTI update kernels tailored for GPU architectures. Update kernels use 1-point updates per work-item/thread or are labeled with (2) to represent 2-point update kernels. One K10 chip runs at 745 MHz and contains 1536 floating point units with 160 GB/s bandwidth. By comparison, the K20x runs at 732 MHz and contains 2496 floating point units with 250 GB/s bandwidth.

Table 3 contains results from two optimized kernels, a CPU-tailored code and a GPU-tailored code, run on OpenMP, OpenCL and CUDA to note performance portability. Although it was expected that optimal CPU-tailored algorithms would not give optimal performance for GPU architectures, we see 40-50% of optimal performance by just running the OCCA kernels in GPU-modes. The GPU-tailored algorithm ran on CPU-modes ended running on 20% performance compared with optimal CPU code, mainly due to the lack of direct control over shared memory as seen on GPU architectures.

	CPU-tailored Kernel	GPU-tailored Kernel
OpenMP	1548	364 (23%)
CUDA (1 K10 core)	515 (41%)	1241
OpenCL (1 K10 core)	665 (51%)	1302

Table 3. Performance comparisons between combinations of OpenMP, CUDA and OpenCL running on the CPU and GPU tailored kernels.

5 Conclusion & future work

We have studied a vertical transverse isotropic propagator discretized with centered finite-differences in time and space. Finite-differences are extensively used in seismic modeling. We have justified the advantage of using high-order stencils both in terms of computational efficiency and points needed per wavelength. To enable the study on various compute architectures, a multi-threaded gateway API to many multi-threading APIs was employed: OCCA. The performance results obtained with the library are generally faster than with the codes written using the best API for the hardware, thanks to the just-in-time compilation. For now, it seems a single OCCA kernel solution performing well for two types of architecture is impossible. The main factor preventing portable optimization is due to the lack of direct control over cache on CPU architectures which can be done on GPU architectures through shared memory. This level of control is

currently only available for GPGPUs and unavailable for traditional CPUs. Having such control on the next generation of CPUs would most certainly re-open possibilities of a single code performing efficiently on both architectures.

We are currently working on implementing features that will make OCCA more accessible to programmers. The current OCCA project is focusing on the OCCA Kernel Language (OKL) derived from compiler-tools to allow for a more native kernel language as opposed to the current macro-based language. A full parser is used for adding language features such as loop-interchange flexibility, automatic OpenCL-CUDA translation to OKL, kernel splitting from multiple outer-loops with the goal to embedded the OKL kernels in regular code.

References

1. R. M. Alford, K. R. Kelly, and D. Mt. Boore. Accuracy of finite-difference modeling of the acoustic wave equation. *Geophysics*, 39(6):834–842, 1974.
2. Charles Cerjan, Dan Kosloff, Ronnie Kosloff, and Moshe Reshef. A nonreflecting boundary condition for discrete acoustic and elastic wave equations. *Geophysics*, 50(4):705–708, 1985.
3. X Du, RP Fletcher, and PJ Fowler. A new pseudo-acoustic wave equation for vti media. In *70th EAGE Conference & Exhibition*, 2008.
4. Bengt Fornberg. The pseudospectral method: Comparisons with finite differences for the elastic wave equation. *Geophysics*, 52(4):483–501, 1987.
5. Bengt Fornberg. Classroom note:calculation of weights in finite difference formulas. *SIAM Review*, 40(3):685–691, 1998.
6. Olav Holberg. Computational aspects of the choice of operator and sampling interval for numerical differentiation in large-scale simulation of wave phenomena. *Geophysical prospecting*, 35(6):629–655, 1987.
7. J. D. McCalpin. Stream: Sustainable memory bandwidth in high performance computers. Technical report, University of Virginia, Charlottesville, Virginia, 1991-2007. A continually updated technical report. http://www.cs.virginia.edu/stream/.
8. David S. Medina, Amik St.-Cyr, and T. Warburton. OCCA: A unified approach to multi-threading languages. *CoRR*, abs/1403.0968, 2014.
9. Paulius Micikevicius. Gpu performance analysis and optimization. In *GPU Technology Conference, http://developer. download. nvidia. com/GTC/PDF/GTC2012/PresentationPDF/S0514-GTC2012-GPU-Performance-Analysis. pdf*, 2012.
10. John Nickolls, Ian Buck, Michael Garland, and Kevin Skadron. Scalable parallel programming with cuda. *Queue*, 6(2):40–53, March 2008.
11. OpenMP Architecture Review Board. OpenMP application program interface version 3.0, May 2008.
12. John E. Stone, David Gohara, and Guochun Shi. Opencl: A parallel programming standard for heterogeneous computing systems. *IEEE Des. Test*, 12(3):66–73, May 2010.
13. Dmitry Vishnevsky, Vadim Lisitsa, Vladimir Tcheverda, and Galina Reshetova. Numerical study of the interface errors of finite-difference simulations of seismic waves. *Geophysics*, 79(4), 2014.
14. Samuel Williams, Andrew Waterman, and David Patterson. Roofline: an insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009.